



Computer Science Department
Web Application and Technologies (COMP 334)
Second Semester 2025/2026

Assignment 3: CSS Styling

Due Date: 25/06/2026

1. Overview

This assignment is a continuation of Assignment 2, in which you built a fully dynamic, database-driven Palestinian souvenir store using PHP and MySQL. In this assignment, you will enhance the visual appearance of all pages produced by the application by writing and applying CSS rules.

Important Constraints:

- **Single external CSS file:** `style.css` only. No embedded `<style>` tags and no inline `style` attributes are permitted anywhere in your HTML output.
- **Course CSS only:** CSS frameworks such as Bootstrap, Tailwind, or any other external library are **not allowed**.
- **Clean CSS file:** Remove any rule or class not actually used in your HTML pages.
- **No JavaScript:** Consistent with Assignment 2 constraints.

2. Linking Your Stylesheet

Add the following line inside the `<head>` element of `header.inc.php` so that every page automatically includes the stylesheet:

```
<link rel="stylesheet" href="style.css">
```

Place `style.css` in the root of your `assThree/` folder alongside your PHP files.

Task 1 — Overall Page Layout (CSS Grid)

Use `display: grid` on a top-level wrapper element that encloses the entire page body (between header and footer). Apply a **two-column** structure:

- **Left column:** sidebar panel (used on `products.php` for the search/filter form).
- **Right column:** main content area (product grid, forms, basket table, etc.).

On pages that do not have a sidebar (e.g. `register.php`, `login.php`, `basket.php`, `checkout.php`), the main content area should **span both columns**.

Task 2 — Header Styling

Style the `<header>` element generated by `header.inc.php`.

Requirements:

- Apply a **background colour** that reflects the Palestinian souvenir store theme (e.g. a warm earth tone or olive green).
- Display the store logo (`<img>`) and the store name (`<h1>`) **side by side**, centred horizontally, using Flexbox.
- Style the `<nav>` element inside the header:
 - Display navigation links **horizontally** using Flexbox.
 - Add consistent **padding** and **spacing** between links.
 - Apply a **hover effect** that changes the link background colour or underline.
 - If a user is logged in, style the displayed name and role with a distinct colour or font style to distinguish it from navigation links.

Task 3 — Footer Styling

Style the `<footer>` element generated by `footer.inc.php`.

Requirements:

- Use Flexbox to arrange the footer content in **two columns**:
 - **Left:** store address (the `<address>` element).
 - **Right:** contact information (telephone, email, and Contact Us link).
- Apply a background colour that complements the header.
- Ensure the footer always appears at the bottom of the viewport regardless of page content length (use appropriate CSS positioning or layout strategy).
- Style the `<address>` element with a legible font style (italic is acceptable but not required).

Task 4 — Search / Filter Panel Styling (Flexbox)

Style the `<aside>` element used as the search panel on `products.php`.

Requirements:

- Arrange the form elements **vertically** using Flexbox.
- Apply consistent **padding** and **spacing** between:
 - Text inputs (Product Name, Max Price).
 - The `<select>` drop-down (Category).
 - The Filter submit button.
 - The Reset / Show All link.
- Give the panel a **light background** (e.g. a soft grey or cream) with a subtle border to visually separate it from the product area.
- Style input fields and the select element with a **consistent width**, border, and padding for a clean look.
- Style the Filter button with a distinct background colour and a **hover effect**.

Task 5 — Product Card Grid (CSS Grid + Flexbox)

5.1 Product Grid Layout

Replace the table-based product listing with a **card-based grid layout** using CSS Grid on the `<main>` element in `products.php`.

Requirements:

- Define a grid with **five columns** and **three rows**:
 - **Rows 1 and 2:** Product cards (up to 10 cards, 5 per row).
 - **Row 3:** Pagination bar (see Task 6).
- Apply consistent gap spacing between cards.
- The grid must display **exactly 10 products per page**.

Figure 1 below illustrates the expected page layout and product grid structure.

FIGURE 1 — PAGE LAYOUT: TWO-COLUMN CSS GRID (SIDEBAR + PRODUCT GRID)

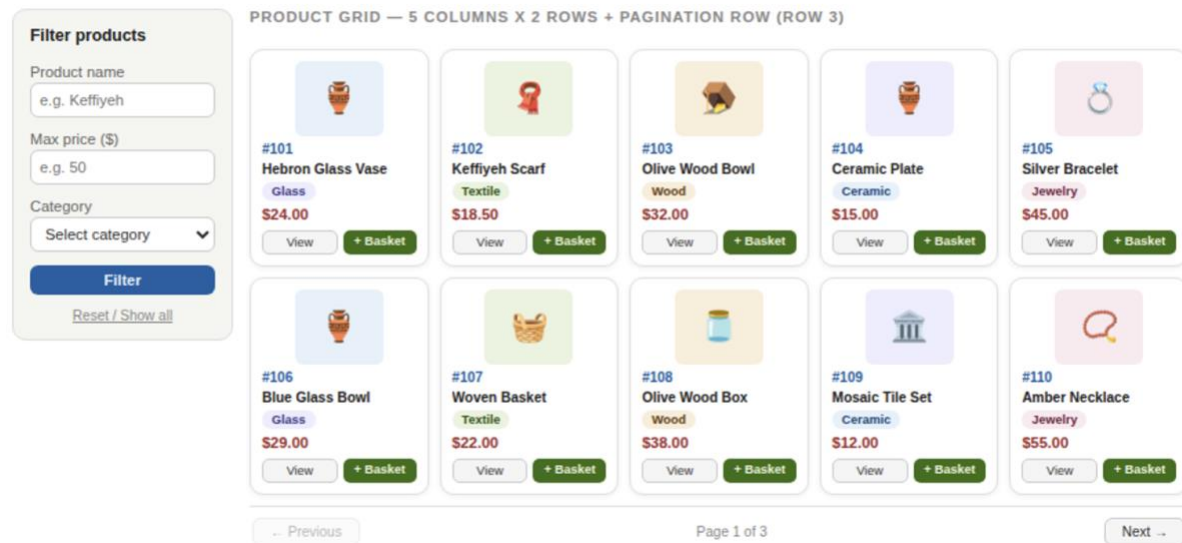
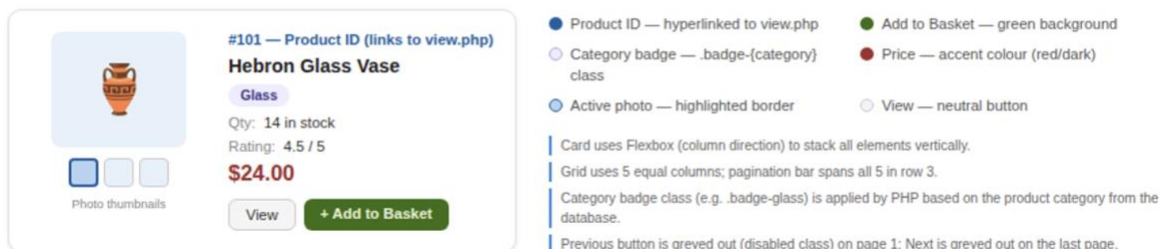


Figure 2 — Product card anatomy



5.2 Product Card Styling

Each card must:

- Use Flexbox to arrange its internal elements **vertically**.
- Have a fixed width that fits neatly within the five-column grid.
- Include a visible **border**, **rounded corners** (`border-radius`), and a subtle **box shadow**.
- Apply a **hover effect** that slightly elevates the card (e.g. increased shadow).

Each card must display the following elements in order, top to bottom:

Element	Styling Requirement
Product image ()	Fixed size; centred horizontally within the card.
Product ID	Displayed as a heading; hyperlinked to <code>view.php</code> .
Product Name	Bold; truncate with ellipsis if too long.
Photo thumbnails	A horizontal row of three small thumbnail images (<code>photo1</code> , <code>photo2</code> , <code>photo3</code>) displayed below the main product image using Flexbox. The currently selected default photo must be visually highlighted (e.g. a coloured border or increased opacity). Clicking a thumbnail links to <code>view.php?id={product_id}</code> .
Category badge	Colour-coded background, rounded pill shape. PHP must apply a CSS class per category (e.g. <code>class="badge badge-ceramic"</code>). Define one rule per category in <code>style.css</code> — see badge class requirements below.
Price	Highlighted colour (e.g. red or dark orange).
Action buttons bar (<nav>)	Horizontal Flexbox row; "View" button in neutral colour; "Add to Basket" button in green.

Category Badge Classes

Define a base `.badge` class in `style.css` for shared properties (font size, padding, `border-radius: 999px`, `font-weight`), then one modifier class per category for the colours. For example:

```
.badge-ceramic {
  background-color: #E6F1FB;
  color: #0C447C;
}
```

Requirements: Define one `.badge-{category}` class for every category present in your database. The colour scheme is **your own choice** — select background and text colours that are visually distinct per category, readable (sufficient contrast), and consistent with the overall site palette. Every category in your data must have a badge class, and any class not used in the HTML must be removed from `style.css`.

Task 6 — Pagination Bar Styling

Style the `<nav>` element used as the pagination bar, placed in the third row of the product grid.

Requirements:

- Use Flexbox to position the **Previous** button on the left and the **Next** button on the right, with the **page indicator** (e.g. "Page 2 of 4") centred between them.
- The pagination bar width must match the full width of the product grid.
- Both buttons must have:
 - Padding for a comfortable clickable area.
 - `border-radius` for rounded corners.
 - A hover effect that changes the background colour.

- Buttons that are not applicable (e.g. Previous on page 1) must be **visually disabled** (greyed out, reduced opacity) — use a CSS class applied by PHP when the button should be inactive.

Task 7 — Registration Form Styling (register.php)

Style the registration form produced by `register.php`.

Requirements:

- Centre the form on the page within the main content area.
- Give the form a **maximum width** (e.g. 600 px) so it does not stretch across the full screen.
- Style the `<fieldset>` and `<legend>` elements:
 - Apply a border and padding to the fieldset.
 - Style the legend as a bold, slightly larger label.
- Arrange each label–input pair in a **two-column row** using Flexbox or Grid:
 - Left cell: the `<label>`, right-aligned.
 - Right cell: the `<input>` or `<select>`, taking the remaining width.
- Apply consistent padding and border styling to all input fields and the select element.
- Highlight **invalid or error fields** with a red border (use a CSS class applied by PHP when validation fails).
- Style server-side error messages in red, positioned directly below the relevant field.
- Style the submit button with a prominent background colour and **hover effect**.

Task 8 — Product Detail Page Styling (view.php)

Style the product detail page rendered by `displayProductPage()`, see assignment 2 for the function specification.

Requirements:

- Display the product detail form in a **centred card** with a border, rounded corners, and box shadow, similar in style to the product cards in Task 5.
- Arrange the three product photos as a **horizontal row** using Flexbox, with the currently selected (default) photo visually highlighted (e.g. a coloured border).
- Style disabled form fields with a greyed-out background to clearly indicate they are read-only.
- Style the action buttons (Back to Products, Add to Basket, Save Changes) consistently with the button styles defined for the rest of the site.

Task 9 — Basket and Checkout Page Styling

9.1 Basket Page (basket.php)

- Style the basket table with clear row borders, **alternating row background colours**, and a header row with a distinct background.
- Right-align numeric columns (Unit Price, Quantity, Line Total).
- Display the **basket total** in a larger, bold font below the table.

- Style the Continue Shopping and Proceed to Checkout links as **buttons**, consistent with the site's button style.

9.2 Checkout Page (checkout.php)

- Apply the same centred-card layout used for the registration form (Task 7).
- Style the credit card form fields consistently with the registration form.
- Style the Place Order submit button with a prominent **green background** and hover effect.

Placement and Submission

File Placement on CS Host

- Place `style.css` in `public_html/ass/assThree/`.
- All PHP pages from Assignment 2 must now be updated to include `header.inc.php` (which links `style.css`) and must produce semantic HTML that your CSS rules can target.
- Update your assignments index page (`ass/index.html`) with a link to `assThree/products.php`.

ZIP Archive

Compress all Assignment 3 files — including the updated PHP files, `style.css`, any updated SQL file, and all images — into a single archive named `assThree-stID.zip` and submit it to ITC before the due date.

Submission Deadline: 25/06/2026 at 22:00 (CS Host server closes at this time). Work will be graded only if correctly submitted to **both CS Host and ITC**. No late submissions will be accepted.